same number, two answers

Python says 2. JavaScript says 3.

```
# Python
>>> round(2.5)
2

// JavaScript
> Math.round(2.5)
3
```

Alvin Wachira

swipe →

last week: store money as integers

Integers fix floats.

They don't fix rounding.

$19.99 stored as **1999** solves the 0.1 + 0.2 problem.

It does nothing for these three:

01 splitting

02 rounding rules

03 formatting

KSh 1,000 shared three ways.

```
>>> 1000 // 3
333
>>> 333 + 333 + 333
999
```

One shilling just disappeared.

give the leftover to someone on purpose

```
[334, 333, 333]
```

Halves don't round the same way everywhere.

input	Python round()	JS Math.round()
0.5	0	1
1.5	2	2
2.5	2	3
3.5	4	4

Python sends halves to the nearest even number (banker's rounding). JavaScript pushes them up.

1.005 was never 1.005.

```
> (1.005).toFixed(2)
"1.00"

// what the float actually holds
1.00499999999999893418589 63...
```

And currencies have their own rules.

Stripe, Ugandan shillings

5 UGX → amount: 500

the decimals must always be 00

save this for your next payments PR

Put this in the README.

01 One rounding mode, named.

02 Round once, at one defined step. Not inside every function.

03 When money is split, the leftover goes somewhere on purpose.

04 Test the ugly numbers: 0.5, 1.005, 1,000 ÷ 3.

Alvin Wachira

which rule does yours use?